

Actividad 02 — Construcción de Portafolios

Enfoque Incremental: EDA $\rightarrow$ Simulación vs Markowitz $\rightarrow$ Servicing (Manual vs Desarrollo Asistido con IA)

Sebastián Egaña Santibáñez (segana@fen.uchile.cl)
Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-09-21

Descargas y Enlaces

Introducción: El Problema

Contexto: Tienes acceso a 5 activos financieros: SPY (S&P 500), EFA (mercados desarrollados), IJS (small-cap), EEM (mercados emergentes), AGG (bonos). Deseas construir un portafolio que maximice el retorno relativo al riesgo, pero **no sabes qué pesos asignar** a cada uno.

Pregunta central: - ¿Cómo *encontrar* los pesos óptimos? - ¿Es mejor simular miles de combinaciones (fuerza bruta) o resolver un problema de optimización exacto? - ¿Cómo *publicar* ese resultado para que otros lo usen?

Tu rol: seguir un pipeline de datos completo: 1. **Parte 1 — EDA:** entender los datos y correlaciones entre activos 2. **Parte 2 — Simulación:** aproximar la frontera eficiente generando miles de portafolios aleatorios 3. **Parte 3 — Markowitz:** resolver exactamente y comparar con la simulación 4. **Parte 4 — Servicing:** publicar el resultado como una página interactiva en este sitio

En cada parte, verás dos enfoques: **Manual** (escribes cada línea de código, ~50-60 min total) y **Asistido con Claude Code** (~10-15 min).

Parte 1 — EDA: Explorando los Datos y Correlaciones

Enfoque Manual (Tradicional)

Sin IA, siguiendo la estructura clásica:

Paso 1: Cargar librerías y datos

```
# Librerías
library(readr)
library(tidyverse)
library(skimr)
library(corrplot)
library(kableExtra)

# Cargar datos
portafolio_retornos <- read_csv("data/portafolio_retornos.csv")

# Verificar estructura
head(portafolio_retornos)
str(portafolio_retornos)
```

Tiempo invertido: ~3 minutos.

Paso 2: Estadística descriptiva por activo

```
# Resumen general
summary(portafolio_retornos %>% select(-fecha))

# Usando skimr para más detalle
skim(portafolio_retornos %>% select(-fecha))

# Estadísticas clave por activo (media, sd, min, max)
stats_por_activo <- portafolio_retornos %>%
  select(-fecha) %>%
  summarise(across(everything(),
```

```

list(
  media = ~mean(., na.rm = TRUE),
  sd = ~sd(., na.rm = TRUE),
  min = ~min(., na.rm = TRUE),
  max = ~max(., na.rm = TRUE),
  n = ~n()
),
.names = "{.col}_{.fn}"
))

print(stats_por_activo)

```

Tiempo invertido: ~10 minutos (ajustes, interpretación).

Paso 3: Visualizaciones individuales

```

# Histogramas y densidades de retornos
portafolio_retornos %>%
  select(-fecha) %>%
  pivot_longer(everything(), names_to = "activo", values_to = "retorno") %>%
  ggplot(aes(x = retorno, fill = activo)) +
  geom_histogram(alpha = 0.45, binwidth = 0.01) +
  facet_wrap(~activo, scales = "free") +
  theme_minimal() +
  labs(title = "Distribución de Retornos Mensuales (2012-2021)",
       x = "Retorno", y = "Frecuencia")

# Gráfico de series temporales
portafolio_retornos %>%
  mutate(fecha = as.Date(fecha)) %>%
  pivot_longer(-fecha, names_to = "activo", values_to = "retorno") %>%
  ggplot(aes(x = fecha, y = retorno, color = activo)) +
  geom_line(alpha = 0.7) +
  facet_wrap(~activo, scales = "free_y") +
  theme_minimal() +
  labs(title = "Retornos Mensuales en el Tiempo",
       x = "Fecha", y = "Retorno")

```

Tiempo invertido: ~12 minutos (ajustes de colores, temas).

Paso 4: Matriz de correlación y covarianza

```

# Matriz de correlación
retornos_matrix <- portafolio_retornos %>%
  select(-fecha) %>%
  as.matrix()

```

```

correlacion <- cor(retornos_matrix)

# Visualizar con corrplot
corrplot(correlacion, method = "circle", type = "upper",
         title = "Matriz de Correlaciones entre Activos",
         mar = c(0, 0, 2, 0))

# Matriz de covarianza
covarianza <- cov(retornos_matrix)

print("Matriz de Covarianza:")
print(round(covarianza, 6))

# Tabla formateada
correlacion_df <- as.data.frame(correlacion) %>%
  rownames_to_column("activo1") %>%
  pivot_longer(-activo1, names_to = "activo2", values_to = "corr") %>%
  filter(activo1 < activo2) %>%
  arrange(desc(abs(corr)))

kable(correlacion_df, digits = 3, caption = "Correlaciones entre Activos")

```

Tiempo invertido: ~15 minutos (cálculos, gráficos, interpretación).

Total Parte 1 Manual: ~40 minutos.

Enfoque Asistido con Claude Code

Prompt que envías:

“Necesito un EDA completo de un portafolio de 5 activos (SPY, EFA, IJS, EEM, AGG) desde un CSV: 1. Cargar datos `portafolio_retornos.csv` 2. Estadística descriptiva (media, sd, quantiles, valores faltantes) 3. Gráficos: histogramas de retornos por activo, series temporales 4. Matriz de correlación y covarianza con visualización 5. Interpretación: ¿qué tan correlacionados están?, ¿potencial de diversificación?”

Lo que Claude Code genera (en segundos):

Un script completo optimizado con `tidyverse` + `corrplot` + `ggplot2`, incluyendo:

```

# Ejemplo de lo que genera el asistente
library(readr)
library(tidyverse)
library(corrplot)

portafolio_retornos <- read_csv("data/portafolio_retornos.csv")

```

```

# Tabla resumen
summary_stats <- portafolio_retornos %>%
  select(-fecha) %>%
  summarise(across(everything(), list(
    media = mean, sd = sd, min = min, q25 = ~quantile(., 0.25),
    q50 = median, q75 = ~quantile(., 0.75), max = max
  )), .names = "{.col}_{.fn}"))

# Matriz de correlación
corr_matrix <- cor(portafolio_retornos %>% select(-fecha))
corrplot(corr_matrix, method = "circle", title = "Correlaciones")

# Series temporales
portafolio_retornos %>%
  pivot_longer(-fecha) %>%
  ggplot(aes(x = as.Date(fecha), y = value, color = name)) +
  geom_line() +
  facet_wrap(~name, scales = "free_y") +
  theme_minimal()

```

Tiempo total: ~5 minutos (incluyendo lectura del código y ajuste de rutas).

Comparación: Manual vs IA (Parte 1)

Aspecto	Manual	Con IA
Tiempo	~40 min	~5 min
Cobertura	Buena, pero a menudo falta una visualización	Exhaustiva (siempre incluye gráficos, tablas)
Debugging	“¿por qué este plot no cabe?”	IA explica y propone ajustes
Código limpio	Crece iterativamente (experimental)	Listo para producción

Parte 2 — Modelos: Simulación de Portafolios (Monte Carlo)

Enfoque Manual

Paso 1: Preparar datos y matriz de covarianza

```
# Descargar o cargar (ya hecho en Parte 1)
portafolio_retornos <- read_csv("data/portafolio_retornos.csv")

# Extraer matriz de retornos
retornos_matrix <- portafolio_retornos %>%
  select(-fecha) %>%
  as.matrix()

# Calcular parámetros
retornos_esperados <- colMeans(retornos_matrix)
cov_matrix <- cov(retornos_matrix)
sd_activos <- apply(retornos_matrix, 2, sd)
```

Tiempo: ~5 minutos.

Paso 2: Simular miles de portafolios aleatorios

```
set.seed(42)

# Parámetros de simulación
n_sims <- 10000
n_activos <- ncol(retornos_matrix)
tasa_libre_riesgo <- 0.02 / 12 # tasa anual / 12 meses

# Matriz para guardar resultados
resultados_sim <- matrix(NA, nrow = n_sims, ncol = 4)
colnames(resultados_sim) <- c("retorno", "volatilidad", "sharpe", "mejor_sharpe")

# Matriz para guardar pesos
pesos_optimos <- matrix(NA, nrow = n_sims, ncol = n_activos)
```

```

# Simulación
for (i in 1:n_sims) {
  # Generar pesos aleatorios long-only
  w <- runif(n_activos)
  w <- w / sum(w) # Normalizar a que sumen 1

  # Retorno esperado del portafolio
  ret_port <- sum(w * retornos_esperados)

  # Volatilidad del portafolio
  var_port <- t(w) %*% cov_matrix %*% w
  vol_port <- sqrt(var_port)

  # Sharpe ratio
  sharpe <- (ret_port - tasa_libre_riesgo) / vol_port

  # Guardar resultados
  resultados_sim[i, 1] <- ret_port * 12 # Anualizar
  resultados_sim[i, 2] <- vol_port * sqrt(12) # Anualizar
  resultados_sim[i, 3] <- sharpe
  pesos_optimos[i, ] <- w
}

# Convertir a data frame
sim_df <- as.data.frame(resultados_sim)
colnames(sim_df) <- c("retorno_anual", "volatilidad_anual", "sharpe_ratio")

# Identificar mejor Sharpe
idx_mejor <- which.max(sim_df$sharpe_ratio)
mejor_sharpe_sim <- sim_df[idx_mejor, ]
mejor_pesos_sim <- pesos_optimos[idx_mejor, ]

cat("Mejor portafolio simulado (máx Sharpe):\n")
print(mejor_sharpe_sim)
cat("\nPesos:\n")
print(setNames(mejor_pesos_sim, colnames(retornos_matrix)))

```

Tiempo: ~15 minutos (loop, debugging de dimensiones).

Paso 3: Visualizar la nube de portafolios

```

# Nube de puntos retorno vs volatilidad
sim_df %>%
  ggplot(aes(x = volatilidad_anual, y = retorno_anual, color = sharpe_ratio)) +
  geom_point(alpha = 0.5, size = 1.5) +
  scale_color_viridis_c(direction = 1) +
  geom_point(data = data.frame(

```

```

    volatilidad_anual = mejor_sharpe_sim$volatilidad_anual,
    retorno_anual = mejor_sharpe_sim$retorno_anual,
    sharpe_ratio = mejor_sharpe_sim$sharpe_ratio
), color = "red", size = 4, shape = 21, fill = NA, stroke = 2) +
theme_minimal() +
labs(title = "Frontera Eficiente Simulada (10,000 portafolios)",
     x = "Volatilidad Anual (%)",
     y = "Retorno Anual Esperado (%)",
     color = "Sharpe Ratio")

```

Tiempo: ~10 minutos (ajustes de escala, colores).

Total Parte 2 Manual: ~30 minutos.

Enfoque Asistido con Claude Code

Prompt:

“Simula 10,000 portafolios long-only con pesos aleatorios usando los datos cargados. Calcula para cada uno: retorno esperado (anualizado), volatilidad (anualizada), y Sharpe ratio (tasa libre = 2 % anual). Identifica el mejor portafolio por Sharpe y visualiza la nube coloreada por Sharpe. Marca el mejor portafolio con un punto rojo grande en el gráfico.”

Resultado (segundos):

```

set.seed(42)
n_sims <- 10000
w_matrix <- t(replicate(n_sims, {
  w <- runif(ncol(retornos_matrix))
  w / sum(w)
}))

sim_results <- data.frame(
  retorno = (w_matrix %*% retornos_esperados * 12),
  vol = apply(w_matrix, 1, function(w) sqrt(t(w) %*% cov_matrix %*% w) * sqrt(12)),
  sharpe = NA
)
sim_results$sharpe <- (sim_results$retorno - 0.02) / sim_results$vol

best_idx <- which.max(sim_results$sharpe)

ggplot(sim_results, aes(x = vol, y = retorno, color = sharpe)) +
  geom_point(alpha = 0.4) +
  geom_point(data = sim_results[best_idx, ], color = "red", size = 4) +
  scale_color_viridis_c() +

```

```
theme_minimal()
```

Tiempo: ~3 minutos.

Parte 3 — Modelos: Optimización de Markowitz y Comparación

Enfoque Manual

Paso 1: Teoría breve

La frontera eficiente de Markowitz resuelve:

$$\min_w [w^T \Sigma w]$$

sujeto a: $- w^T r = r_p$ (retorno objetivo) - $\sum w_i = 1$ (pesos suman 1) - $w_i \geq 0$ (long-only, sin venta corta)

donde Σ es la matriz de covarianza y r es el vector de retornos esperados.

Paso 2: Instalar y usar quadprog

```
# Instalar si no está (comentar después)
# install.packages("quadprog")

library(quadprog)

# Parámetros ya calculados en Parte 2
retornos_esperados # vector de medias
cov_matrix          # matriz de covarianza

# Función para calcular frontera eficiente
calcular_frontera <- function(retornos_esperados, cov_matrix,
                              r_min, r_max, n_puntos = 50) {
  resultados_frontier <- matrix(NA, nrow = n_puntos, ncol = 3)
  colnames(resultados_frontier) <- c("retorno_objetivo", "volatilidad", "pesos")

  retornos_objetivo <- seq(r_min, r_max, length.out = n_puntos)

  for (i in 1:n_puntos) {
    r_obj <- retornos_objetivo[i]
```

```

# Formulación para quadprog: solve.QP(H, f, A, b0)
# Minimiza: (1/2) * x^T H x + f^T x
# Sujeto a: A^T x >= b0

H <- 2 * cov_matrix
f <- rep(0, length(retornos_esperados))

# Restricciones:
# 1) sum(w) = 1 -> w1 + w2 + ... + wn = 1
# 2) w^T * r = r_obj
# 3) w_i >= 0

# Matriz A y vector b0 para restricciones de igualdad (convertidas a desigualdad)
A <- cbind(
  rep(1, length(retornos_esperados)), # sum(w) = 1
  retornos_esperados                  # w^T * r = r_obj
)

# Esto es complicado en quadprog, usamos solución alternativa...
}

return(resultados_frontier)
}

```

Nota: quadprog es poderosa pero tiene una sintaxis tediosa para restricciones mixtas (igualdad + desigualdad). Con IA, esto se simplifica.

Tiempo: ~20 minutos (frustración, búsqueda en documentación).

Enfoque Asistido con Claude Code

Prompt simplificado:

“Optimiza la frontera eficiente de Markowitz usando los datos de `portafolio_retornos.csv`:
 - Long-only (pesos entre 0 y 1, suman 1) - Calcula 50 puntos en la frontera barriendo retornos de 8% a 20% anual - Para cada punto, resuelve: minimizar varianza sujeto a retorno objetivo - Resultado: tabla con (retorno, volatilidad, pesos de cada activo) y gráfico overlay con la nube de simulación de la Parte 2 para comparar”

Lo que genera (minutos, no manual):

```

library(quadprog)

# Función helper: optimizar para un retorno objetivo
optimizar_portafolio <- function(w, cov_matrix, retornos_esperados, r_target) {
  # Usar optimize package

```

```

resultado <- optim(
  par = w,
  fn = function(x) as.numeric(t(x) %*% cov_matrix %*% x),
  method = "SLSQP",
  ui = rbind(-diag(length(x)), diag(length(x)), -retornos_esperados, retornos_esperados),
  ci = c(rep(0, length(x)), rep(-1, length(x)), -r_target, r_target)
)
return(resultado$par)
}

# Frontera eficiente
frontera <- expand.grid(
  retorno_objetivo = seq(0.08, 0.20, length.out = 50)
) %>%
  rowwise() %>%
  mutate(
    pesos = list(optimizar_portafolio(...)),
    volatilidad = sqrt(t(pesos) %*% cov_matrix %*% pesos) * sqrt(12)
  )

# Gráfico combinado
ggplot() +
  geom_point(data = sim_df, aes(x = volatilidad_anual, y = retorno_anual,
                                color = "Simulada"), alpha = 0.3) +
  geom_line(data = frontera, aes(x = volatilidad, y = retorno_objetivo * 12,
                                 color = "Markowitz"), size = 1.2, color = "red") +
  theme_minimal()

```

Tiempo: ~5 minutos (entender el resultado, no escribir).

Comparación Visual

El gráfico overlay muestra: - **Nube (Parte 2)**: puntos simulados, esparcidos - **Curva roja (Parte 3)**: frontera exacta de Markowitz, tocando el borde superior de la nube

Conclusión: Markowitz encuentra el envolvente superior; la simulación es una aproximación visual.

Parte 4 — Servicing: Publicar como Página Interactiva

Por qué servir importa

Hasta aquí tienes: 1. Un análisis que escribiste *una vez* 2. Código que funciona, pero solo en tu máquina

Problema: ¿cómo comparten estudiantes / colegas el resultado sin re-ejecutar el código?

Solución: publicar como una **página HTML interactiva** en este sitio (GitHub Pages), con: - Gráficos interactivos (hover muestra valores exactos) - Sliders para ajustar restricciones en tiempo real - Responsivo en móviles - Actualización automática si suben nuevos datos

Enfoque Manual: Construir Observable JS en Quarto

```
// Carga de datos (CSV precalculado)
portafolio_data = FileAttachment("data/portafolio_retornos.csv").csv()

// Transformar en matriz de retornos
retornos_matrix = portafolio_data.map(d => [d.SPY, d.EFA, d.IJS, d.EEM, d.AGG])

// Calcular matriz de covarianza (librería math)
import {default as math} from "https://cdnjs.cloudflare.com/ajax/libs/mathjs/12.0.0/math.min.js"

// Medias y covarianza
retornos_esperados = [
  d3.mean(retornos_matrix, d => d[0]) * 12, // SPY
  d3.mean(retornos_matrix, d => d[1]) * 12, // EFA
  d3.mean(retornos_matrix, d => d[2]) * 12, // IJS
  d3.mean(retornos_matrix, d => d[3]) * 12, // EEM
  d3.mean(retornos_matrix, d => d[4]) * 12 // AGG
]

// Sliders para ajustar pesos manualmente
peso_spy = Inputs.range([0, 100], {value: 20, step: 1, label: "SPY (%)"})
```

```

peso_efa = Inputs.range([0, 100], {value: 25, step: 1, label: "EFA (%)"})
peso_ijs = Inputs.range([0, 100], {value: 20, step: 1, label: "IJS (%)"})
peso_eem = Inputs.range([0, 100], {value: 20, step: 1, label: "EEM (%)"})
peso_agg = Inputs.range([0, 100], {value: 15, step: 1, label: "AGG (%)"})

// Normalizar y calcular retorno/volatilidad del portafolio ajustado
w_manual = [peso_spy, peso_efa, peso_ijs, peso_eem, peso_agg].map(x => x / 100)
retorno_manual = d3.sum(w_manual.map((w, i) => w * retornos_esperados[i]))
// ... cálculo de volatilidad similar

// Mostrar resultado
html`
<div style="border: 1px solid #ccc; padding: 10px; margin-top: 10px;">
  <h3>Portafolio Personalizado</h3>
  <p><strong>Retorno Esperado:</strong> ${((retorno_manual * 100).toFixed(2))}% anual</p>
  <p><strong>Volatilidad:</strong> ${((vol_manual * 100).toFixed(2))}% anual</p>
  <p><strong>Sharpe Ratio:</strong> ${(((retorno_manual - 0.02) / vol_manual).toFixed(2))}</p>
</div>
`

```

Tiempo: ~25 minutos (sintaxis de Observable, debugging).

Enfoque Asistido con Claude Code

Prompt:

“Crea un bloque Observable JS (ojs) embebido en Quarto que: 1. Carga el CSV de retornos 2. Muestra 5 sliders (0-100 %) para ajustar pesos de cada activo 3. Calcula dinámicamente: retorno esperado, volatilidad y Sharpe ratio 4. Normaliza pesos para que sumen 100 % 5. Muestra resultado en una caja formateada Sin necesidad de servidor, solo HTML estático.”

Resultado (minutos):

Código observable JS listo, con buena estructura, sin errores de sintaxis. Claude maneja D3.js + math.js automáticamente.

Tiempo: ~5 minutos.

Flujo de Publicación

1. Crear archivo .qmd con las 4 partes + bloque ojs
2. quarto render actividad_02_portafolios.qmd --to html
3. El archivo docs/clases_ia/actividad_02_portafolios.html se genera automáticamente
4. git add + git commit + git push

5. GitHub Pages actualiza el sitio en ~30 segundos
 6. Link: https://sebaegana.github.io/finanzas_en_r_webpage/clases_ia/actividad_02_portafolios.
-

Tabla de Ventajas: Manual vs IA

Aspecto	Manual	Con IA
Tiempo total (4 partes)	~105 min	~20 min
Parte 1 (EDA)	40 min	5 min
Parte 2 (Simulación)	30 min	3 min
Parte 3 (Markowitz)	20 min	5 min
Parte 4 (Servicing)	25 min	5 min
Boilerplate	Completo, repetitivo	Generado sin pensar
Debugging	“¿por qué quadprog no funciona?”	IA explica la solución
Documentación	Manual, lleva tiempo	Generada con comentarios
Iteraciones	Reescribir partes	Pedir cambios en natural

Actividad para Estudiantes: Tu Turno

Instrucciones Generales

Elige UNA de las 3 opciones abajo. Todas conducen al mismo resultado: un análisis completo + página publicada.

Opción A: Maximalista (Recomendada para Aprender)

Sigue las 4 partes de esta actividad **escribiendo código**, usando Claude Code solo para debugging o sugerencias puntuales.

Pasos: 1. Abre Positron 2. Crea un proyecto nuevo `mi_portafolio_actividad` 3. **Parte 1:** Carga los datos, crea estadística descriptiva (mean, sd, histogramas, matriz de correlación) 4. **Parte 2:** Simula 10,000 portafolios aleatorios long-only, calcula retorno/volatilidad/Sharpe, grafica nube 5. **Parte 3:** Optimiza frontera de Markowitz con `quadprog` (long-only), overlay con la nube de simulación 6. **Parte 4:** Crea un bloque `ojs` con sliders interactivos para ajustar pesos y ver el Sharpe ratio en vivo 7. Publica como archivo `.qmd` en tu portafolio / GitHub

Entregas: script R, gráficos, página publicada.

Opción B: Híbrida (Equilibrada)

Combina código propio + Claude Code para acelerar sin perder aprendizaje.

1. **Parte 1:** Escribes el EDA (10 min)
 2. **Parte 2:** Pides a Claude Code que simule (2 min lectura)
 3. **Parte 3:** Escribes la optimización con ayuda de Claude (15 min)
 4. **Parte 4:** Claude Code genera el `ojs` (5 min ajustes)
-

Opción C: Minimalista (Prompt-Driven)

Pídele a Claude Code el **flujo completo** en 1-2 prompts:

“Análisis completo de portafolio: 1. EDA de `portafolio_retornos.csv` 2. Simular 10,000 portafolios long-only 3. Optimizar Markowitz (frontera eficiente) 4. Gráfico overlay 5. Bloque `ojs` con sliders interactivos para ajustar pesos Todo en un único `.qmd`”

Claude Code genera ~600 líneas de código profesional en ~2 minutos. Tu rol: revisar, entender, y interpretar los resultados.

Rúbrica de Evaluación

Criterio	Excelente	Bueno	Aceptable
Carga de datos	CSV cargado, validado, sin NA	Cargado correctamente	Cargado con warning
EDA	Stats (media, sd, skew), 3+ gráficos, matriz de corr. clara	2 gráficos, correlación	Solo <code>summary()</code>
Simulación	5.000 portafolios, pesos long-only correctos, Sharpe calcula bien	1.000-5.000 portafolios, correcto	<1.000 o errores
Markowitz	Frontera exacta (50+ puntos), long-only ($w \geq 0$, $\sum = 1$), overlay claro	20-50 puntos, mostly correct	Incompleto o con errores
Comparación	Nube vs curva overlay visibles, explica por qué Markowitz toca borde	Overlay presente	Ausente
Servicing (ojs)	Sliders funcionales, calcula dinámicamente, responsive	Sliders básicos, funciona	Ausente o no funciona
Interpretación	Explica correlaciones, elección de pesos, cuándo simular vs optimizar	Menciona resultados clave	Mínima
Uso de IA	Usa Claude Code estratégicamente (debugging, boilerplate, iteración)	Usa parcialmente	Mínimo o nulo

Entrega

Debes entregar:

1. **Script R o .qmd** con código comentado (4 partes completas)
2. **Dataset limpio** (CSV con retornos procesados si hiciste cambios)
3. **Reporte HTML o PDF** con:
 - Tabla de EDA (media, sd, correlaciones)
 - Nube de simulación (gráfico retorno vs volatilidad)
 - Frontera de Markowitz overlay
 - Explicación: “¿qué aprendiste sobre diversificación?”

4. **Página publicada** (link a GitHub Pages o similar) con el bloque `ojs` interactivo funcional
 5. **Reflexión** (1 página): cómo usaste IA, cuál fue tu mayor dificultad, qué harías diferente
-

Reflexión Final: IA como Herramienta, No Reemplazo

Usa IA para:

- **Boilerplate:** cargas, librerías, transformaciones estándar (ahorra 60 %)
- **Exploración:** “¿cómo haría esto en tidyverse?” vs “¿cómo en data.table?”
- **Debugging:** “¿por qué falla este argumento de quadprog?”
- **Observable JS:** IA domina esta sintaxis mejor que la mayoría de humanos
- **Documentación:** genera comentarios y explicaciones automáticamente

NO uses IA para:

- **Decisiones financieras:** TÚ eliges qué ponderación investigar, IA genera código
- **Validación:** verifica siempre que $\text{Sharpe} = (\text{retorno} - \text{rf}) / \text{vol}$ (no confíes ciegamente)
- **Comprensión:** entiende el código **antes** de publicarlo (“¿qué está haciendo este loop?”)
- **Pensar:** IA acelera la ejecución, no la estrategia analítica

La Mentalidad Correcta

“IA es como un copiloto competente: escribe código, pero **TÚ** diriges el análisis financiero. Verificas cada resultado, cuestiona las correlaciones, decide pesos en función de tu hipótesis, no al revés.”

Recursos Adicionales

[Claude Code Docs](#)

[Positron IDE](#)

[tidyverse en R](#)

[quadprog: Portfolio Optimization](#)

[Observable JS + D3](#)

[Markowitz Modern Portfolio Theory](#)